

“HỌC ĐỂ LÀM VIỆC”

C++ for Base

Contents

1

Variable, data type Advanced in C++

2

Method in C++

3

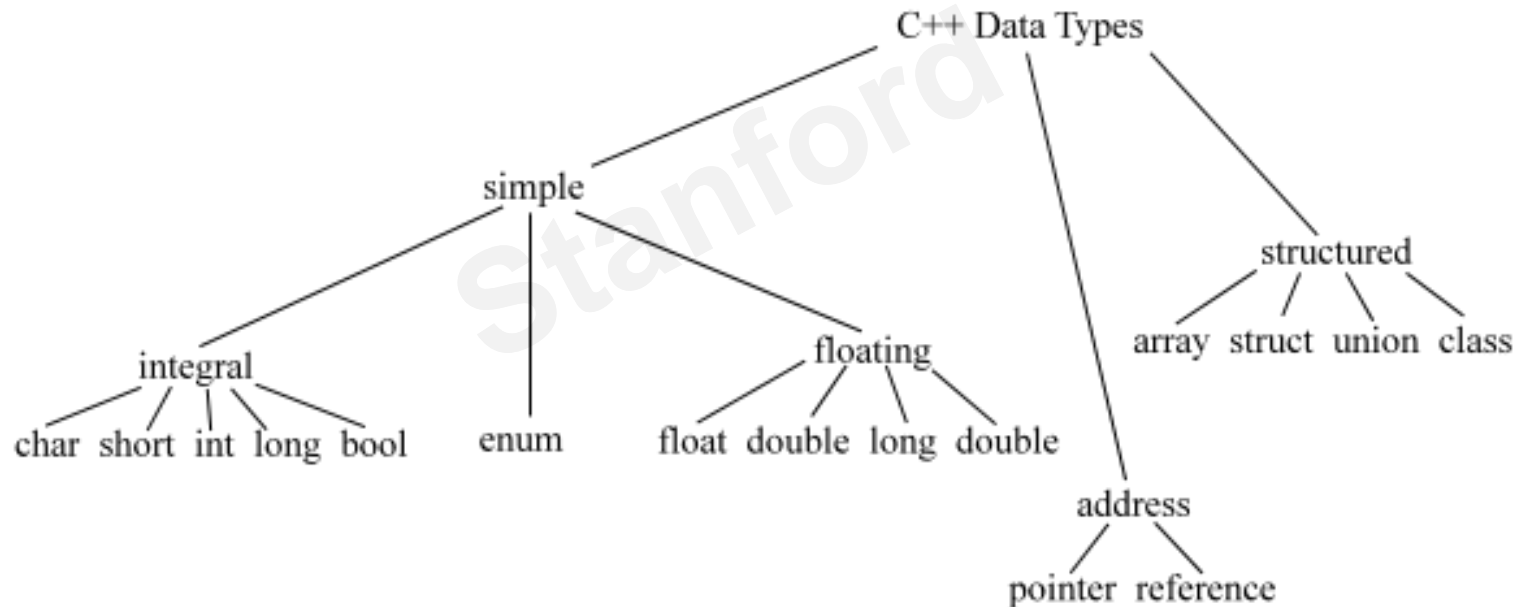
Dynamic Memory

Variable, data type Advanced

C++ for Base

❖ Variable, data type Advanced in C++

- Trong C++ chia ra 3 kiểu biến
 - Kiểu giá trị; Kiểu con trỏ; Kiểu tham chiếu



C++ for Base

❖ Variable, data type Advanced in C++

- Trong C++ chia ra 3 kiểu dữ liệu
 - Kiểu giá trị
 - Kiểu con trỏ (pointer)
 - Kiểu tham chiếu

❖ Variable, data type Advanced in C++

- **Kiểu giá trị:** Là kiểu dữ liệu bao gồm char, short, int, long, bool,...khi người dùng khai báo sử dụng máy tính sẽ cấp phát cho một vùng nhớ để lưu trữ giá trị của biến đó.
 - Ví dụ: `int tuoi = 35; char hoten[20] = "Nguyen Hai Anh";`

❖ Variable, data type Advanced in C++

- **Kiểu con trỏ (pointer):** Là kiểu dữ liệu giá trị của biến này chứa địa chỉ của biến khác.

– Ví dụ:

» `int x=20;`

» `int *p;`

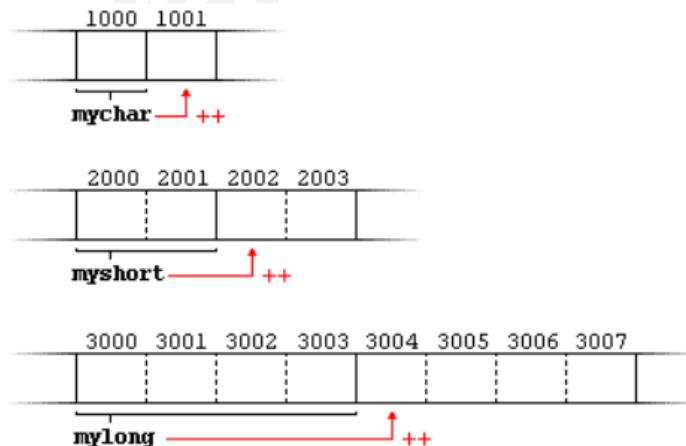
» `p = &x` // Khi đó p chứa giá trị là địa chỉ của biến x là: 0xbfc601ac

❖ Variable, data type Advanced in C++

■ Kiểu con trỏ (pointer)

- Khai báo: Kiểu dữ liệu **<*tên_biến>**
- Sử dụng kí hiệu ***** trước tên biến khai báo hoặc sau kiểu dữ liệu

Ví dụ: `int *p;` //Khai báo biến p trỏ đến kiểu dữ liệu nguyên



❖ Variable, data type Advanced in C++

■ Kiểu con trỏ (pointer)

- Để con trỏ p trỏ đến biến x ta phải dùng phép gán $p = \text{địa chỉ của } x$.
 - Nếu x không phải là mảng ta viết: $p = \&x$.
 - Nếu x là mảng ta viết: $p = x$ hoặc $p = \&x[0]$.
- Không gán p cho một hằng địa chỉ cụ thể. Ví dụ viết $p = 20$ là sai. Phép toán $*$ cho phép lấy nội dung nơi p trỏ đến, ví dụ để gán nội dung nơi p trỏ đến cho biến f ta viết $f = *p$.
- $\&$ và $*$ là 2 phép toán ngược nhau. Cụ thể nếu $p = \&x$ thì $x = *p$. Từ đó nếu p trỏ đến x thì bất kỳ nơi nào xuất hiện x đều có thể thay được bởi $*p$ và ngược lại.

❖ Variable, data type Advanced in C++

■ Kiểu con trỏ (pointer)

Xét một số loại con trỏ sau:

- **C++ NULL pointers:** Khai báo này được sử dụng khi con trỏ không trỏ đến địa chỉ cụ thể. Ví dụ: `int *ptr = NULL;`
- **C++ pointer arithmetic:** Sử dụng các toán tử ++, --, +, - để trỏ đến địa chỉ tương ứng.

❖ Variable, data type Advanced in C++

■ Kiểu con trỏ (pointer)

- **C++ pointer arithmetic:** Sử dụng các toán tử ++, --, +, - để trỏ đến địa chỉ tương ứng.

Ví dụ:

```
int arr[MAX] = {10, 100, 200};
```

```
int *ptr;
```

```
ptr = arr;
```

Sau đó duyệt vòng lặp tăng biến con trỏ lên `ptr++` in ra các giá trị, địa chỉ của con trỏ p.

❖ Variable, data type Advanced in C++

■ Kiểu con trỏ (pointer)

- **Array of pointers:** Khai báo mảng là dạng con trỏ trong đó các phần tử của mảng là con trỏ chứa địa chỉ của giá trị trỏ đến.

Ví dụ:

```
const int MAX = 3; int arr[MAX] = {10, 100, 200}; int *ptr[MAX];  
for (int i = 0; i < MAX; i++)  
    { ptr[i] = &arr[i]; //gán địa chỉ của các phần tử mảng var.}  
for (int i = 0; i < MAX; i++){cout << "Value of arr[" << i << "] = "; cout  
<< *ptr[i] << endl; }
```

❖ Variable, data type Advanced in C++

■ Kiểu con trỏ (pointer)

- **Return pointer from functions:** Là giá trị trả về của một hàm có kiểu con trỏ

Ví dụ:

```
int * getRandom( )  
{static int r[10]; srand( (unsigned)time( NULL ) );  
  for (int i = 0; i < 10; ++i) {r[i] = rand(); cout << r[i] << endl;  
  }  
  return r;} //Sử dụng hàm srand cần khai báo #include <ctime>
```

❖ Variable, data type Advanced in C++

■ Kiểu con trỏ (pointer)

- **Return pointer from functions:** Là giá trị trả về của một hàm có kiểu con trỏ

Ví dụ:

```
int *p;  
p = getRandom();  
for ( int i = 0; i < 10; i++ ) {  
    cout << "(p + " << i << " ) : ";  
    cout << *(p + i) << endl; }
```

❖ Variable, data type Advanced in C++

- **Kiểu tham chiếu (Reference):** Là một bí danh (alias) của một biến kiểu giá trị nào đó và nó sử dụng bộ nhớ của biến này.
- Biến tham chiếu không được cấp phát bộ nhớ, không có địa chỉ riêng.

Ví dụ: float u, v, &r=u; tạo ra các biến thực u, v và biến tham chiếu thực r.

- Biến tham chiếu phải được khai báo rõ ràng tham chiếu đến biến nào cụ thể

❖ Variable, data type Advanced in C++

■ Kiểu tham chiếu (Reference):

- Biến tham chiếu phải được khai báo rõ ràng tham chiếu đến biến nào cụ thể:
 - Ví dụ: `double &x; //Bị lỗi`
- Biến tham chiếu có thể tham chiếu đến 1 phần tử của mảng
 - Ví dụ: `int a[5], &r= a[2];`
- Không cho phép khai báo mảng tham chiếu
- Biến tham chiếu có thể tham chiếu đến 1 hằng. Sử dụng vùng nhớ của hằng và có thể làm thay đổi giá trị của vùng nhớ này.
 - Ví dụ:
`&r = 68;`
`r++; cout <<r; // in ra số 69`

❖ Variable, data type Advanced in C++

■ Kiểu cấu trúc (struct):

- Sử dụng để tạo ra một biến có cấu trúc theo định nghĩa của người dùng.
- **Cú pháp:**

```
struct Ten_kieu_cau_truc  
{  
    //Khai báo các biến trong kiểu cấu trúc  
}
```

❖ Variable, data type Advanced in C++

■ Kiểu cấu trúc (struct):

- Sử dụng để tạo ra một biến có cấu trúc theo định nghĩa của người dùng.

Ví dụ:

```
struct ThiSinh
{
    char hoten[25];
    long sobaodanh;
    float toan, ly, hoa, tongdiem;
}
```

❖ Variable, data type Advanced in C++

■ Kiểu cấu trúc (struct):

- Khai báo biến kiểu cấu trúc

- **<tên cấu trúc> <danh sách biến> ;//** trong C++

Các biến được khai báo cũng có thể đi kèm khởi tạo:

- **<tên cấu trúc> biến = { giá trị khởi tạo } ;**

Ví dụ: ThiSinh ts1, ts2 = {"Nguyen Minh Anh", 2688, 7, 8, 9}

❖ Variable, data type Advanced in C++

■ Kiểu cấu trúc (struct):

- Truy nhập các thành phần kiểu cấu trúc
 - Đối với biến thường: **tên biến.tên thành phần**
 - Đối với biến con trỏ: **tên biến → tên thành phần**
 - Đối với cấu trúc lồng nhau. Truy nhập thành phần ngoài rồi đến thành phần của cấu trúc bên trong, sử dụng các phép toán.

Method in C++

❖ Method in C++

Hàm là một chương trình con trong chương trình lớn. Hàm giúp cho việc phân đoạn chương trình thành những module riêng rẽ, hoạt động độc lập. Một hàm có thể được sử dụng trong chương trình này mà cũng có thể được sử dụng trong chương trình khác, để cho việc kiểm tra và bảo trì chương trình.

- Nằm trong hoặc ngoài văn bản có chương trình gọi đến hàm. Được gọi từ chương trình chính (main), từ hàm khác hoặc từ chính nó (đệ quy).
- **Có 3 cách truyền giá trị:** Truyền theo tham trị, tham biến và tham trở.

❖ Method in C++

- Cú pháp khai báo:

```
<kiểu hàm> <tên hàm>(danh sách tham đối hình thức)  
{  
    //Viết code xử lý công việc trong nội bộ hàm  
    (return (biểu thức trả về))  
}
```

- Trong đó:

- **kiểu hàm**: là giá trị trả về của hàm hoặc hàm không có giá trị trả về dùng từ khóa **void**.
- **return**: giá trị trả về sau lời gọi hàm

❖ Method in C++

Ví dụ: Hàm thực hiện công việc có giá trị trả về

```
int TongSo(int a, int b)
{
    int tongso=0; tongso = a + b;
    return tongso;
}

int PhepNhan(int a, int b)
{
    int tich=0;
    tich = a * b;
    return tich;
}
```


❖ Method in C++

Ví dụ: Hàm không có giá trị trả về chỉ thực hiện công việc

```
void InDanhSach(int n)
{
    for(int i=0; i<n; i++)
    {
        cout << "So thu tu la "<<i;
    }
}
```

❖ Method in C++

Ví dụ: Hàm tính lũy thừa của x^n

```
double luythua(float x, int n)
{
    int i ; // khai báo biến chỉ số
    double kq = 1 ; // khai báo biến để lưu kết quả
    for (i=1; i<=n; i++) kq*= x ;
    return kq;
}
```

❖ Method in C++

▪ Hàm với đối số mặc định

Trong quá trình khai báo các đối số trong hàm sẽ được gán giá trị mặc định.

Cú pháp khai báo:

<kiểu hàm> **<tên hàm>**(đ1, ..., đn, đmđ1 = gt1, ..., đmđm = gtm) ;

Ví dụ: Gọi hàm tính lũy thừa như sau:

```
void XoaManHinh(int n=100) { int i;  
  
    for (i=1; i<=n; i++) clrscr();  
  
    return ; } //Gọi hàm XoaManHinh()
```

❖ Method in C++

▪ Nạp chồng các hàm (Overloading method)

Trong C++ cho phép người lập trình có thể đặt các hàm có tên giống nhau nhưng khác nhau về kiểu giá trị trả về và đối số truyền vào.

Ví dụ: Hàm tính tổng 2 số như sau:

```
int Tong(int a, int b);
```

```
long Tong(long a, long b);
```

```
double Tong(double a, double b);
```

❖ Method in C++

▪ Truyền theo tham trị

Là cách khai báo truyền thống, quá trình thực hiện hàm sẽ cấp phát vùng nhớ để lưu trữ giá trị của các biến trong hàm. Sau lời gọi hàm các thông tin này sẽ được giải phóng và có thể trả về giá trị nếu hàm có giá trị trả về.

Ví dụ:

```
void HoanDoi(int x, int y)
{
    int t ; t = x ; x = y ; y = t ;
}
```

❖ Method in C++

▪ Truyền theo con trỏ

Là cách khai báo sử dụng để thay đổi các giá trị sau lời gọi hàm

Ví dụ:

```
void HoanDoi(int *p, int *q)
{
    int t; // khai báo biến tạm t
    t = *p ; // đặt giá trị của t bằng nội dung nơi p trỏ tới
    *p = *q ; // thay nội dung nơi p trỏ bằng nội dung nơi q trỏ
    *q = t ; // thay nội dung nơi q trỏ tới bằng nội dung của t
}
```

❖ Method in C++

▪ Truyền theo tham chiếu

Là cách khai báo sử dụng để thay đổi giá trị của các tham số sau lời gọi hàm bằng cách khai báo hàm với các đối số ở dạng tham chiếu.

Ví dụ:

```
void HoanDoi(int &x, int &y)
{
    int t = x; x = y; y = t;
}
```

❖ Method in C++

▪ LỜI gọi và sử dụng hàm

Là cách người lập trình gọi hàm ở một nơi nào đó để thực hiện công việc trong hàm được mô tả.

Cú pháp khai báo:

- **tên hàm**(danh sách tham đối thực sự);

Danh sách tham đối thực sự còn gọi là danh sách giá trị gồm các giá trị cụ thể để gán lần lượt cho các đối hình thức của hàm.

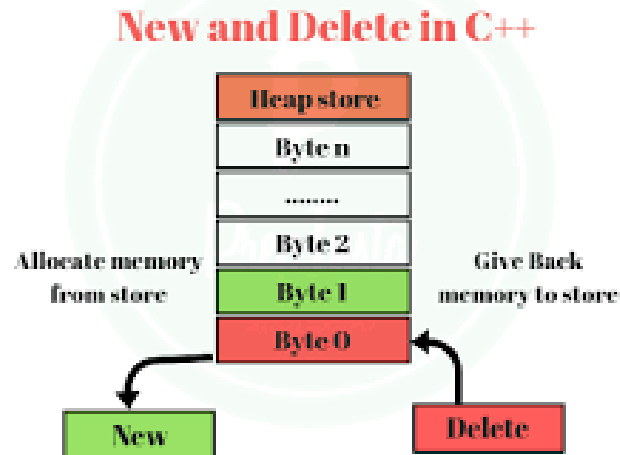
Ví dụ: Gọi hàm tính lũy thừa như sau:

```
int x=5, n=2; luythua(x,n) //Giá trị 25
```


Dynamic Memory

❖ Cấp phát bộ nhớ động trong C++

- Đối với lập trình c++ để cấp phát vùng nhớ chúng ta sẽ sử dụng từ khoá new và giải phóng sử dụng từ khoá delete



❖ Cấp phát bộ nhớ động trong C++

▪ Ví dụ 1: Cấp phát bộ nhớ động cho biến

```
//Khai báo biến sử dụng vùng nhớ động
int* pr1 = new int;

//Gán giá trị
*pr1 = 8;

cout << "Gia tri cua pr1 la: " << *pr1 << endl;

//Giải phóng vùng nhớ
delete pr1;
```

❖ Cấp phát bộ nhớ động trong C++

- **Ví dụ 1:** Khai báo 1 mảng cấp phát bộ nhớ động

```
//Khai báo mảng chứa 2 phần tử  
int* pr = new int[2];
```

```
//Gán giá trị cho các phần tử  
pr[0] = 5;  
pr[1] = 6;
```

❖ Cấp phát bộ nhớ động trong C++

▪ Ví dụ 1: Khai báo 1 mảng cấp phát bộ nhớ động

```
//In các phần tử
cout << "new: Các phần tử của mảng là: " << endl;

for (int i = 0; i < 2; i++)
{
    cout << pr[i] << endl;
}

//Giải phóng vùng nhớ sau khi sử dụng
delete[]pr;
```

“HỌC ĐỂ LÀM VIỆC”

Thank You !